

Prioritization

Julius Götsch and Isabel Seidl

University of Potsdam, Am Neuen Palais 10; 14469 Potsdam; Germany
goetsch@uni-potsdam.de, seidl1@uni-potsdam.de

1 Introduction

Any working pathfinding encoding for Flatland in Answer Set Programming (ASP) guarantees that every train reaches its destination on time, provided that at least one answer set exists and no malfunctions occur. Otherwise, it would simply return unsatisfiable. Optimization statements can be added to reduce unnecessary waiting or detours. However, within these constraints, the decisions made to resolve conflicts between trains, such as which train proceeds and which must wait or take a detour, are still determined arbitrarily. This report aims to investigate which aspects can influence these decisions and how they can be used to achieve a desired outcome. We understand prioritization as the deliberate choice to favor a certain train when deciding which one must wait or take an alternative route. We outline where and when prioritization becomes relevant and describe how we implemented it in ASP. Furthermore, we demonstrate how prioritization can be leveraged to optimize schedules for robustness against malfunction-related delays, and we provide an outlook on how prioritization could be applied in a synchronous simulation setting.

2 Assigning priorities

The priority a train has will be represented by a number. The bigger the number the higher priority of a train. In the following section, we present three approaches to assigning priorities to trains.

2.1 System-Assigned Priorities

Starting off we envisioned a number of fixed priority groups each with an associated priority value. Each train would then be assigned to one of these groups. This assignment would be chosen randomly or using a modulo-based method for even distribution. One could interpret the priority groups as different types of trains with a priority order between groups and unchanged behaviour within the same group. While this worked on a technical level, this method is very rigid and has limited use in practical applications.

2.2 User-Assigned Priorities

Alternatively, priorities may be set according to the user’s preferences to simulate a specific scenario. To achieve this, users may manually set priority values for each train. This could be done to simulate different train types as groups again (e.g. freight and passenger trains) or set priorities individually with any integer value. Notably, the absolute priority values are not so important, what matters are the relative priority differences to other trains.

2.3 Environment-Assigned priorities

Instead of directly coming up with a specific number as a priority the idea with this approach is to calculate the priority value based on variables from the environment. Apart from their ID trains can be differentiated and thus prioritized differently based on several environment variables. Such variables could include starting location, location of destination, departure time, deadline, distance to destination, or any other numerical value that can be interpreted from the environment. The chosen variables can then be combined in a formula to calculate the priority value. Which variables and formula to choose depends on the goal one wants to achieve through prioritization. We generally focused our efforts on maximizing the robustness of our solution in regard to malfunctions (more on that in section 4), but for the purpose of this approach, it could be any goal. In the following, we outline an example of what this could look like.

First let’s define the environment variables: **time window** and **distance** to its destination.

X/Y Distance: The x/y distance of a train is measured as the absolute difference between the X and Y coordinates of the train’s starting and ending cells. The x/y distance of a train is the same in all answer sets. It is important to note that this coordinate-based distance only approximates the actual steps the train must take. In certain environments, relying on this type of approximation can be misleading or even detrimental, as it may significantly misrepresent the true travel path. Figure 1 illustrates this limitation.

$$\text{Distance}_T = |X_{T\text{End}} - X_{T\text{Start}}| + |Y_{T\text{End}} - Y_{T\text{Start}}| \quad (1)$$

Time Window: In Flatland, each train is assigned a personal time horizon specified by the **end** fact. Although in Flatland, these time horizons are just preferences - in our context, we treat it as binding. The time window of a train represents the time available for a train to reach its destination. It is calculated as the difference between the deadline and spawning time. The Time Window of a train is the same among all answer sets.

One might want to put the distance in relation to the time window since a train with a short time window relative to the distance it must cover has a tighter

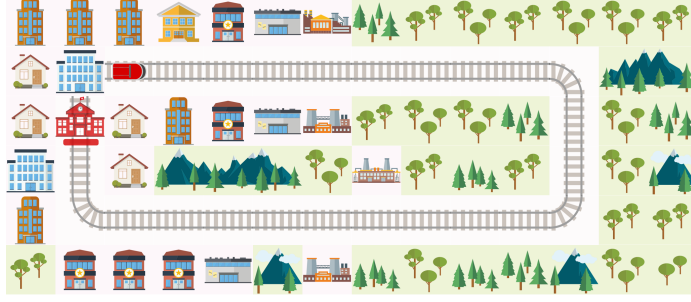


Fig. 1. example where the coordinate-based distance fails to represent the actual travel duration

schedule. Conversely, if a train has ample time to cover the required distance, its priority could be low. With this in mind, a train's priority could be determined as shown in equation (2).

$$\text{Priority}_T = \frac{\text{Distance}_T}{\text{Time Window}_T} \quad \text{with} \quad \text{Time Window}_T \geq \text{Distance}_T \quad (2)$$

With this, a bigger distance as well as a smaller time window will increase the priority value of a train. Notably, because in Flatland (without speed) a train will always take one timestep for one unit of distance the Distance will always be smaller than or equal to the Time Window, and the Priority is always smaller than or equal to one. But since Clingo operates only with integer calculations, it cannot directly represent fractions in ASP. As a straightforward solution, we multiply the priority equation by a constant factor of 1000, effectively shifting three decimal places to the left. This results in three-digit priority values within Clingo. For greater accuracy, this factor can be increased, at the cost of larger absolute priority values.

3 Prioritization in practice

Having assigned different priorities to trains, the next natural follow-up question is how these priorities change interactions of trains in the environment. For that, we first want to outline when and where this matters.

3.1 Types of conflicts

There are several different situations in Flatland where trains interfere with one another and where prioritization determines the outcome. We will refer to these situations as conflicts. Note that this differs slightly from the traditional flatland terminology where conflict refers to trains directly occupying the same cell or swapping positions. Those occurrences should already be eliminated by any working pathfinding encoding.

The simplest form of a conflict in our case is a **single wait conflict** where two trains arrive at an intersection at the same time. This results in one train having to wait to let the other one pass first. Which train gets to go first can be decided by prioritization. Generally, we differentiate between conflicts that require trains to **wait** to be resolved and conflicts that require trains to take **detours**. A single wait conflict is an example of the former. However, required waiting times can also be significantly longer in the case of track sections that are used bidirectionally. (see Fig. 2)

Fig. 2. environment, where one train must wait on the other. The order is decided by priority

Forced detours, on the other hand, arise when waiting is not an option because the trains would indefinitely block each other's path. An example of this can be seen in Figure 3. The major difference between waits and detours for conflict resolution is that detours also increase the distance traveled, whereas waits do not. This becomes relevant for any prioritization method that takes distance traveled into account.

There may be many variations where both waiting and taking a detour are viable options. In that case, the encoding should naturally choose whichever option takes less time. Also feasible are situations where neither waiting nor taking a detour is possible. Those would be considered deadlocks and are eliminated beforehand by the pathfinding encoding or, if that is not possible, result in unsatisfiable.

Fig. 3. environment with shortcut and detour, where the train to take the detour is decided purely by which priority is greater

3.2 Applying priority

Based on the previous section, the intuitive next step would be to apply the priorities on a per-conflict basis and give the right of way to whichever train has higher priority. However, because the behaviour that is decided on any given conflict ultimately affects the timing of every subsequent conflict, viewing every conflict individually is not advised. Consider the environment depicted in Figure 4.

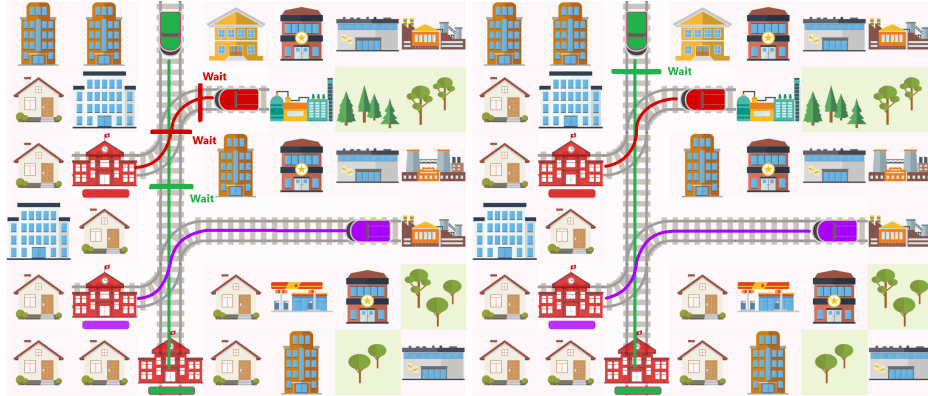


Fig. 4. Illustration of how local prioritization (left) is outperformed by global prioritization (right)

In this example, the green train gets a higher priority assigned than the red train but lower than the purple train. On a per-conflict (local) basis, we would

first evaluate the conflict between the green and the red train. Priority dictates that the green train would pass the first intersection unhindered and only waits for the higher-priority purple train. This would cause the red train to wait twice behind the green train. A global view on prioritization on the other hand could recognize that the green train needs to wait one time anyway, thus letting the red train go first and with that eliminating two unnecessary waits. An observation we made was that the previous example could simply be solved in ASP by optimizing for the total time taken by all trains. Something we had already implemented in our original pathfinding encoding with the statement $\#minimize\{T : end_time(ID, T)\}$, where T is the time it took each train to reach its destination. We refer to this as optimizing for efficiency. It turns out one can realize the desired global prioritization similarly. Since it still needs to be a valid pathfinding solution, the desired prioritization is already among the answer sets produced by the encoding and can theoretically be found with optimization statements.

For that purpose, we initially leveraged the inherent priority syntax in the minimize to realize fixed priority groups. We devised a series of $\#minimize\{T@P : end_time(ID, T), prio(ID, P)\}$ statements where P stands for the respective priority group. Clingo then optimizes for the value before the @-sign in descending priority. Meaning only in case of a tie between answer sets for the highest priority group, the second one would be considered, and so on.

However, not only did we want to stray away from fixed priority groups, but there was another problem with this approach. With it, the highest priority train would always get the shortest possible path. The same goes for the second highest as long as it doesn't impede the one above and so on. While this solves the previous example in Fig. 4 well and it sounds like it is exactly what we want, it turns out hard-enforcing priorities is not always desirable.



Fig. 5. higher priority train (left) forces lower priority train (right) to take arbitrarily long detour

Take the example illustrated in Fig. 5. Giving the higher priority train on the left the guaranteed shortest path forces the other train to take an arbitrarily long detour. Alternatively, the left train could wait a few time steps, letting the right train pass first. While we do want to give the left train an advantage, one can imagine that at a certain point, the cost for this on the entire system becomes too high. What we would want is some kind of threshold after which the prioritization gets overruled in favour of efficiency. For this, we devised the statement $\#minimize\{P * T, ID : end_time(ID, T), prio(ID, P)\}$. We are now minimizing the time as before, but weighted by the priority values of each train. With this weighted sum the time taken by a higher priority train contributes more to the cost, achieving the desired prioritization. However, if the cost contributed by a different train becomes too high because its detour or waiting time gets too long, the priority order can get overruled. The same applies when the additional cost is distributed across many lower-priority trains - collectively, their delays may outweigh the benefit of prioritizing a single train.

To illustrate this threshold consider the example in Fig. 6 where one of the two trains must take a detour to resolve the conflict. The lower priority train on the left would have to take a detour of 14 extra steps whereas the higher priority train on the right would only have a detour of 5 additional steps. Assigning priorities based on X/Y distance and time window as proposed earlier, we arrive at priority values 360 for the right train and 281 for the left train respectively. The cost function then determines the cost of the long detour to be too high and grants the low-priority train the right of way. The threshold at which the lower-priority train is forced to take the detour sits at a priority value of 128 or lower.

Fig. 6. environment with detours of different lengths, where the lower priority train (left) takes the shortcut because its detour is longer

4 Leveraging Prioritization for Robustness

In general, Robustness is a system’s ability to handle unexpected events or disruptions. Within the context of train scheduling, robustness encompasses multiple aspects, depending on the degree of flexibility allowed in the scheduling process.

4.1 Fixed schedule robustness

We consider a scenario where train departure and arrival times are predetermined and cannot be altered. We refer to this setting as *fixed schedule*. Such situations are common in freight operations, where trains are loaded with goods that must be delivered within strict time constraints. The primary objective in this setting is to ensure that all trains reach their destinations before their assigned deadlines, despite the possibility of minor malfunctions. First, it is essential to define a few key terms that will be used throughout this section.

Scheduled Travel Duration: The travel duration of a train is defined as the difference between the arrival time and the time of spawning in a planned schedule. Unlike the time window, a train’s arrival time can vary across different answer sets, and so does its scheduled travel duration. This duration includes both the time spent in motion and any planned waiting time. It is important to note that this is the **scheduled** duration; in the presence of malfunctions, the **actual** travel duration may differ.

Slack: A key factor influencing robustness in fixed schedules is the availability of slack, defined as the difference between the allowed time window and the train’s scheduled travel duration. Trains with minimal or no slack are particularly vulnerable, as they lack the temporal flexibility to cope with a delay. Robustness can be substantially improved by avoiding schedules in which trains have little or no slack.

Finding the right approach

Our initial approach to achieving fixed schedule robustness focused on assigning **explicit priorities** to trains based on their remaining distance to the destination (measured as X/Y distance) and the size of their time window relative to that distance, as introduced in section 2.3. This strategy may offer reasonable results in certain scenarios, particularly when:

- The X/Y distance is a reliable approximation of the actual travel path.
- Information about scheduled travel durations is unavailable.

However, this method presents several shortcomings. For instance, assigning the lowest priority to a train solely because it has a relatively short distance to cover

in proportion to its time window - compared to other trains - results in the train being disadvantaged consistently in every conflict situation. Over time, this can exhaust its slack entirely. In practice, scheduled travel duration is a more accurate metric than distance for assessing how tight a train's schedule is in a given answer set. It reflects the actual time a train is scheduled to detour or wait in its given schedule. However, this leads to a paradox with explicit priorities. Let's say a train receives low priority because its scheduled travel duration is small compared to its time window, but precisely this should lead to it being disadvantaged, thus increasing its scheduled travel duration again. It would also mean that a train's priority varies across answer sets.

As a result, we moved away from explicit prioritization and tried to develop a cost function at the level of answer sets. We refer to this approach as implicit prioritization, since it still determines which train is prioritized in conflict situations, but does so indirectly through the optimization of scheduling criteria, rather than fixed per-train priority values.

A natural starting point for this cost function was to consider absolute slack, preferring answer sets in which the total slack across all trains was maximized. However, absolute slack alone is insufficient. To illustrate this, we present the following example:

We assume a time-based malfunction rate (e.g., a train statistically experiences a malfunction every 20 time steps).

Train 1: Allotted 450 time steps to reach its destination, 400 steps to cover

Train 2: Allotted 1050 time steps to reach its destination, 1000 steps to cover

Both trains have a slack of 50. However, assuming a malfunction occurs every 20 time steps, each lasting 1 time step:

Train 1 loses approximately 20 time steps due to malfunctions.

Train 2 loses approximately 50 time steps due to malfunctions.

Although both trains have the same absolute slack time, Train 2 is at a significantly higher risk of missing its deadline. Even in the absence of reliable malfunction statistics, it is reasonable to assume that trains with longer travel durations are statistically more likely to have longer delays. Therefore, our objective became to design a cost function that penalizes answer sets where trains have low slack relative to their scheduled travel duration.

Cost function:

$$\min \sum_{\text{Train ID}} \frac{\text{Scheduled travel duration}}{\text{Slack} + 1} \quad (3)$$

Characteristics of this cost function:

- Heavily punished answer sets with train schedules with long travel duration and little slack
- The division remains valid even when the slack is zero, as 1 is added to the denominator to prevent division by zero.

A trade-off with this approach to increase robustness is oftentimes reduced efficiency, meaning the sum of all scheduled travel durations may not be minimal.

4.2 Adaptive schedule robustness

Unlike fixed schedules, adaptive scheduling provides the flexibility to adjust train departure and arrival times. The key objective in adaptive robustness is to prevent the propagation of minor disruptions.



Fig. 7. Caption

For example, consider a scenario where: Red train is expected to arrive at $t = 10$, Magenta train departs at $t = 10$, and Purple train departs at $t = 11$. If the red train arrives just a few time steps later, it prevents the magenta and purple trains from departing, thereby causing the delay to propagate. Instead, an adaptive schedule enables the insertion of a time buffer in such cases to prevent subsequent delays. If the departure times of the magenta and purple trains included a built-in time buffer of just a few time steps, the propagation of delays would be avoided.

5 Prioritization in a Synchronous Approach**5.1 Introduction**

In this chapter, we examine prioritization within a synchronous simulation. We are considering a scenario where we lack the ability to define a conflict-free schedule before initiating a synchronous simulation. In this case, each agent follows its own independent travel plan, but these plans may not inherently be

collision-free. Such a situation can arise either because no initial schedule was created or because, after a malfunction, there is insufficient time to recalculate a new schedule. As a result, conflicts and deadlocks must be detected and resolved during the simulation.

The challenge of avoiding deadlocks in synchronous simulations has been a long-standing research topic. In [1], the authors present Dynamic Route Reservation (DRR) as a potential solution. We found it particularly relevant because DRR operates on clearly defined rules, making it a promising candidate for implementation using ASP.

5.2 How Dynamic Route Reservation works

The core principle of DRR is that before a train is authorized to enter a track, a predefined number of subsequent track sections must be reserved. The reservation of tracks is realized according to a series of rules, which are included in the appendix along with a detailed example illustrating their application. It is recommended to review this example before proceeding with the main text, as understanding of the original DRR rules is essential to understand the rest of this section.

Station tracks in DRR

In DRR, station tracks are defined as tracks that belong to a station, which consists of at least two parallel tracks. This setup allows trains, first, to wait while other traffic proceeds on a parallel track, and second, to pass trains traveling in the opposite direction. The concept of station tracks is fundamental to deadlock prevention in DRR. In fact, the railway networks presented in [1], consist solely of simple straight tracks and station tracks.

5.3 Adapting DRR to Flatland

Now, we turn to adapting the original Dynamic Route Reservation (DRR) approach to a Flatland setting. At this stage, we aim to establish a conceptual framework that could guide future implementation. In the appendix, we provide a detailed example that references many of the presented rules that do not have examples directly in the main text.

Implementing Stations

We can replicate the concept of stations in Flatland using different track configurations. In the following chapter, we present two possibilities.

In the first configuration (see Fig. 8 left image), we introduce four station tracks (marked in green), where trains can wait without obstructing traffic. In the top three green squares, up to three trains moving in the same direction can wait, while traffic continues to flow through the tracks below. In the lower green

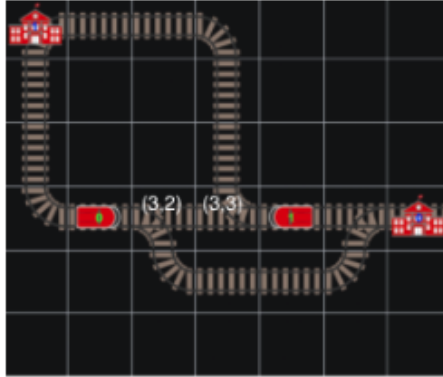
square, a single train can wait while traffic flows freely on the track above. In the second configuration (see Fig. 8 right image), we only have two station tracks. Both setup differs from the station concept in the original DRR paper, which includes two station tracks.



Fig. 8. four station tracks (left), two station tracks (right)

Alternative to Stations: Decision Points

A decision point is a location defined by coordinates and a track type that allows a train, arriving from direction D , to choose between at least two non-waiting actions that would eventually allow it to reach its destination.



In this 6×7 grid, train 0 has a decision point at $(3, 2)$ and train 1 has a decision point at $(3, 3)$. There are no other decision points in this example. We also considered the idea of using decision points instead of stations. Under this approach, trains would not reserve tracks all the way to a station track but instead reserve up until their next decision point. The reasoning behind this is that at a decision point, trains still have the option to take an alternative route to avoid a deadlock or conflict, making it a safe place for them to go to. However, for this work, we decided to first explore the station-based concept as presented in [1].

Discrete Time Steps

In any kind of simulation, time progresses in discrete timesteps. In Flatland, all trains must complete their next action within a timestep. This means that, in DRR, trains not only have to complete their movement within a single time step, but also have to finalize their next set of reservations.

Extending Rule Set

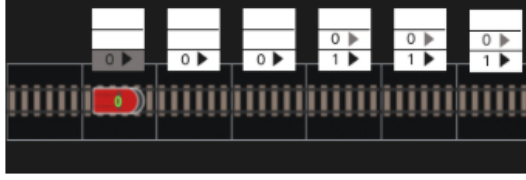
There are at least three reasons why an extension of the rule set is necessary:

1. In Flatland trains spawn at their starting position and despawn at their final destination. To account for this behavior, we must define special reservation rules.
2. We want to apply prioritization to make sure a higher priority train reservation is at the first position in the stack if reservation order impacts its movement.
3. The configuration of the station track as illustrated above.

So - **in addition to the original DRR rules** - we introduce 9 additional rules to accommodate for a Flatland setting:

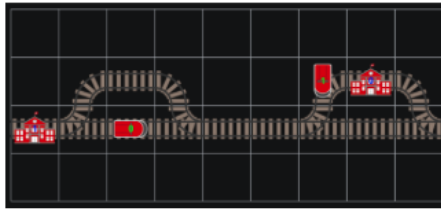
Additional Rules

1. **Reservation before Spawning:** If a train wants to spawn at timestep t , it must successfully reserve its spawning position, along with any necessary reservations that result from this reservation, at timestep $t - 1$. If the reservation fails at $t - 1$, the train cannot spawn at t . Furthermore, the reservation for the spawning position has to be at the top of the stack.
2. **Preventing spawning on track reserved by an opposing train:** A train may never spawn on a track that is reserved by an opposing train. If a train attempts to reserve a track that is reserved by an opposing train, the reservation fails.
3. **Preventing simultaneous spawning of two trains at the same position:** If two trains attempt to spawn at the same position at the same time, the reservation of the higher-priority train is placed at the top of the stack. That means that at that time only the higher priority train can spawn.
4. **Spawning on a position reserved by a follower train:** If Train A wants to spawn at time t while Train B has already reserved the spawning position moving in the same direction at an earlier timestep, and is scheduled to move there at a time later than t , then Train A's reservation can be moved to the top of the stack.



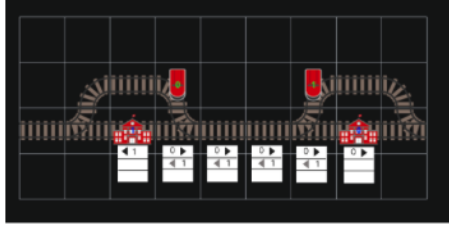
In this example, Train 0 is positioned at $(0, 1)$ and has already made reservations for the entire track segment. At the same time, Train 1 wants to spawn at position $(0, 4)$. At time t , Train 1 successfully makes a reservation for its spawning position. At time $t+1$, it spawns and occupies position $(0, 4)$ at $t+2$. At $t+2$, Train 0 reaches position $(0, 3)$, now one step behind Train 1, following it without experiencing any delay. Train 1 successfully enters the track without disrupting the already-reserved movement of Train 0.

5. **Reservation of top station tracks:** If a train reserves one of the top three station tracks, it must also reserve all other top station tracks of that station along its path. This prevents an opposing train from entering the top station tracks and creating a deadlock.
6. **Extended version of Rule 2:** If a train reserves a track section with bidirectional operation other than a station track, the route to leave that section must also be reserved. The first position outside the bidirectional track section can either be a station track or the train's final destination (despawning position).
7. **Despawning Rule:** If a train has already made reservations up to its final destination, it does not make further reservations in the following timesteps.
8. **Detouring:** If a train fails to make a reservation and alternative routes are available, the train may attempt to reserve an alternative path. This mechanism is particularly important in scenarios where waiting alone cannot resolve conflicts, and a detour is the only working solution.



In this example, both Train 0 and Train 1 attempt to reserve their shortest paths to their respective final destinations, but both fail due to Rule 5. Train 0 has no alternative paths available. Train 1 has an alternative route by taking the top station tracks of the left station. Train 1 successfully reserves this path. Since Train 1 will now exit the bidirectional section at a different location than where Train 0's original route comes from, Train 0 can now successfully stack its reservation directly behind Train 1.

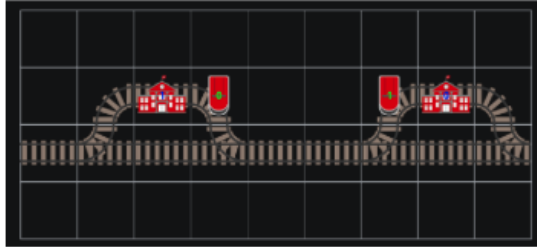
- 9. Opposing reservations:** When two trains attempt to make new reservations for the same position within the same timestep both coming from station tracks, but are approaching from different directions (not following one another), the reservation of the higher-priority train is placed at the top of the stack. This rule results in a situational conflict resolution.



In this 4 x 9 grid example it is clear that either train 0 or train 1 has to wait for the other to leave the bidirectional track segment before it can move along. Both trains attempt to reserve the bidirectional track segment up to their final destination. Assuming train 0 is of higher priority its reservation gets stacked to the top. Train 1's reservation won't get to the top of the stack until train 0 has cleared the last bidirectional track (2, 6).

Counter example

The following example shows that there are still issues with the current rule set.



Currently, no rule prevents train 0 and train 1 from spawning into these positions. While they are not technically in a deadlock, as they are not immediately blocked from moving - they are in a situation where neither can reach its destination.

5.4 Conclusion

It is already clear that this rule set is not final and will require further refinement. Additionally, investigating alternative approaches to synchronous conflict detection and resolution beyond Dynamic Route Reservation could be really interesting. However, we anticipate that many of these approaches may not be compatible with ASP.

Appendix

In the appendix, we first present the rules as introduced in [1], followed by an illustrative example of their application. Finally, we provide a detailed example demonstrating how the extended rule set is applied to a Flatland environment.

Rules

The DRR rules as presented in [1]:

Rule 1: Before a train is authorised to enter a track section, the route into that section must be reserved.

Rule 2: If a route is reserved that leads into a track section with bidirectional operation other than a station track, the route to leave that section must also be reserved.

Rule 3: If a track section to be reserved is still reserved for or occupied by another train, the reservation for that section is stacked to the next position.

Rule 4: If a reservation is put into the stack behind another train, for that train, if not yet the case, the route to leave that section must be reserved.

Rule 5: If a route to be reserved leads into a track that is reserved for or occupied by any opposing train, i.e. a train that will leave that section by moving into the section the route to be reserved comes from, the reservation of that route fails.

Rule 6: A reservation comes only into effect if all following reservations that are initiated by that reservation by rules 2 and 4 are possible. If the reservation of a route fails, all reservations previously initiated by rules 2 and 4 fail, too.

Importantly, a train may only move to a track if:

- The reservation was successful.
- The train holds the first position in the reservation stack for that track.

Example of Rule Application in original DRR

Figure 15 in [1] provides an example of how DRR functions in practice. In the following paragraph, we explain how the rules were applied in this example.

Rule 2 was applied to Trains 5 and 6 because they are on a bidirectional track section. This requires them to also reserve the route needed to exit that section. As a result, both trains have reserved their routes up to Station C. Since Train 6 is stacked behind Train 5, **Rule 4** dictates that Train 5 must

reserve a route to leave that section. Consequently, Train 5 has extended its reservation to Station B. Similarly, because Train 5 is stacked behind Train 4, Train 4 must also reserve its route to leave the section, leading it to reserve a path to Station A. Train 3 also follows **Rule 2**, reserving its route up to Station C. So far, all these reservations have been determined based on the positions of the trains.

Now, Train 1 wants to leave Station A. According to **Rule 1**, a train may only enter a track section for which it has a valid reservation. The track after Station A is already reserved by Train 4, so the reservation for Train 1 is stacked behind Train 4 (**Rule 3**). Applying Rule 2, Train 1 must also reserve a route to leave the bidirectional track segment, which extends its reservation up to Station B. In Station B, Train 1's reservation is stacked behind Train 2. As a consequence of Rule 4, Train 2 must then reserve a route to leave Station B, leading it to extend its reservation up to Station C, where it is stacked behind Train 3. Applying Rule 4 again, Train 3 must then reserve a route to leave its segment, forcing it to extend its reservation to Station D.

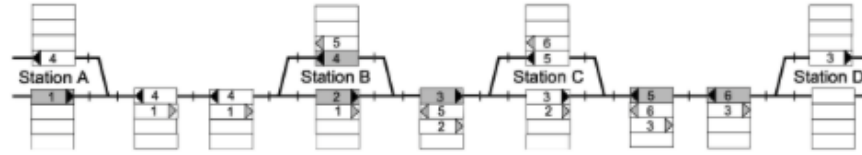


Figure 15: EXAMPLE F

	Section occupied by Train X
	Section reserved at first position for Train X
	Section reserved at stacked position for Train X

Source of images: [1]

Example of Rule Application adapted to Flatland

On the following pages, we examine a 6x11 grid environment used to illustrate a relatively simple scenario involving three trains over three time steps. This example demonstrates how track reservations are made and how subsequent actions are guided by those reservations. References to the original DRR rules are made using their rule numbers (e.g., *Rule 4*), while references to the extended rule set use the rule names (e.g., *Reservation before spawning*). In the accompanying tables, bold positions indicate the final destinations of the trains. Since the tables don't convey the order of the reservation stack, a corresponding image is provided for each time step to visually represent the stack.

At the first time step, Train 1 attempts to reserve position (3, 3). Since this position is part of a bidirectional track segment, it must also attempt to reserve positions (3, 4) and (3, 5). This is required by Rule 2, which states that a train must reserve the route to leave the bidirectional section. If any of these reservations fail, the train can not spawn at (3, 3). Train 2 attempts to reserve position (2, 5), which is the central top station track of the second station. According to the *Reservation of top station tracks rule*, a train must reserve all top station tracks in its direction of travel. Therefore, if Train 2 were unable to reserve (2, 4) in addition to (2, 5), the entire reservation attempt would fail and it would not be able to spawn at (2, 5). In this instance, however, both Train 1 and Train 2 successfully reserve their spawning positions as well as the required subsequent tracks. As a result, they spawn at the second time step. At this time, both trains also continue to reserve tracks along their planned paths, already reserving up to their final destination. Train 3, attempts to reserve position (2, 1), but fails, as this track is already reserved by Train 2 approaching from the opposite direction (Rule: *Preventing spawning on track reserved by an opposing train*). In subsequent time steps, Train 3 may continue attempting to reserve its spawning position, but it won't be successful until Train 2 clears the track. After the second time step, Trains 1 and 2 do not make new reservations, they can proceed along their paths to their destinations.

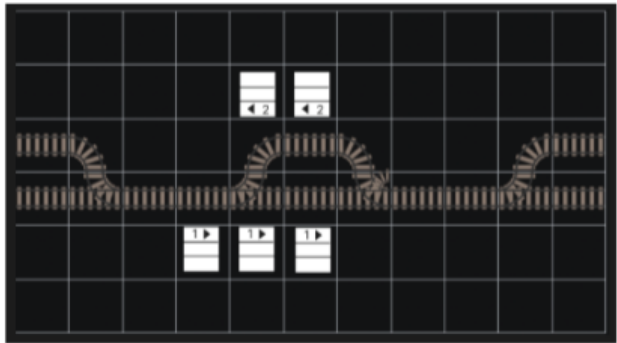
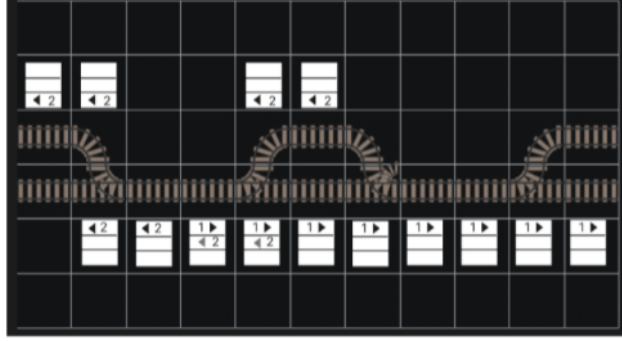


Fig. 9. T= 1

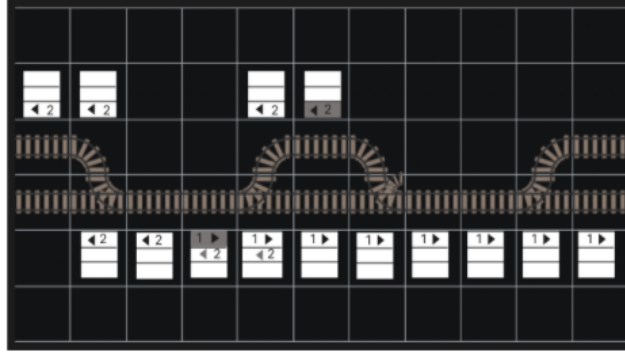
	Train 1	Train 2	Train 3
Position	None	None	None
Action	None	None	None
New reservation request with respective rule	(3, 3) Reservation before Spawning (3, 4), (3, 5) Rule 2	(2, 5) Reservation before Spawning (2, 4) Reservation of top station tracks	None
New reservation request status	Successful	Successful	None
All current reservations	(3, 3), (3, 4), (3, 5)	(2, 5), (2, 4)	None

Table 1. T = 1

Fig. 10. $T = 2$ 

	Train 1	Train 2	Train 3
Position	None	None	None
Action	Spawn	Spawn	None
New reservation request with respective rule	(3, 6) Rule 1 (3, 7), (3, 8), (3, 9), (3, 10) Extended version of Rule 2	(3, 4) Rule 1 (3, 3), (3, 2), (3, 1), (2, 1), (2, 0) Extended version of Rule 2	(2, 1) Rule 1
New reservation request status	Successful	Successful	Failed Preventing spawning on track reserved by an opposing train
All current reservations	(3, 3), (3, 4), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9), (3, 10)	(2, 5), (2, 4), (3, 4), (3, 3), (3, 2), (3, 1), (2, 1), (2, 0)	None

Table 2. $T = 2$

Fig. 11. $T = 3$


	Train 1	Train 2	Train 3
Position	(3, 3)	(2, 5)	None
Action	Move-forward	Move-forward	None
New reservation request with respective rule	None Despawning Rule	None Despawning Rule	(2, 1) Rule 1
New reservation request status	None	None	Failed Preventing spawning on track reserved by an opposing train
All current reservations	(3, 4), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9), (3, 10)	(2, 4), (3, 4), (3, 3), (3, 2), (3, 1), (2, 1), (2, 0)	None

 Table 3. $T = 3$

Acknowledgements

We thank MadMotion for providing the tool *Clingonia* used in the production of the GIFs and images.

References

1. Pachl, J.: Avoiding Deadlocks in Synchronous Railway Simulations. (2007). <https://doi.org/10.24355/DBBS.084-200704030200-0>.